

Object Oriented BDD and Executable Human-language Module Specification

Eric Lee
SE Cloud (Shanghai) Intelligent
Technology Co., Ltd.
Shanghai, China
eric99519@gmail.com

Jiayu Gong
Shanghai Key Laboratory of Computer
Software Testing & Evaluation
Shanghai Development Center of
Computer Software Technology
Shanghai, China
gjy@sscenter.sh.cn

Qinghong Cao
System Operation Department
Shanghai Stock Exchange
Shanghai, China
qhcao@sse.com.cn

Abstract—This paper presents an approach to software development which uses a generative AI Model as compiler to translate human language requirements into high-level programming language. We propose an executable human-language module specification and a tool to support it, which has been used successfully for human-language UI test automation. We anticipate further development of this approach to enable complex software to be programmed in human language, allowing for more intuitive and efficient software development.

Keywords—OOBDD (Object-oriented Behavior Driven Development), GPT, SDD (Specification Driven Development), BDD, HLP (Human Language Programming), LLM (Large Language Model), Generative AI.

I. INTRODUCTION

The success of generative artificial intelligence represented by ChatGPT has brought a huge impact to many fields including software engineering. ChatGPT can write code for us using transformer models [1], given human-language instructions or prompts. For example, giving ChatGPT the following instruction, “Write a Java function to concatenate all strings in a list.”, ChatGPT will write give us an implementation. But most of our software is large and complex. How to tell ChatGPT to write large and complex software, such as a database or a commercial grade eCommerce application? If we can answer this question, we will make good progress in human-language programming.

To write a Java program, we need a programmer, Java programming language, and a compiler. To reduce complexity, we break down a large application into smaller modules; we use object-oriented code structure, such as Java Classes, to help programmers write modularized code.

Similarly, to achieve human-language programming (HLP), we need four elements:

1. A human language such as English to write program statements. We have it already.
2. A Compiler to transform human-language program to a high-level programming language program and test script. This role is filled by generative artificial intelligence models.
3. A compiler to convert the high-level programming language program to machine code. Any IDE will do the job.
4. A universal human language module specification template to break down human-language program into smaller modules, make complex human-language

program understandable by humans and generative artificial intelligence models. We do not have this yet.

We want to propose a solution to the last item, the missing piece of the puzzle. Bertrand Meyers also thinks that the combination of model-based systems and software engineering (MBSSE [2]) and LLM is the key for future software engineering.[3]

Specification-Driven Development (SDD) focuses on creating software that meets a set of predefined specifications. Both the software and the test script for the software can be created from the specification. SDD can be strict or relaxed.

Specification written in formal language is considered as precise specification. [4][5] We can write a software to translate a precise specification to a program. Precise specification may be too difficult and costly.

Many agile software development methodologies, such as TDD (Test-Driven development [6]), BDD (Behavior Driven Development [7][8]) are examples of relaxed SDD. Their specifications are not designed for code generation and full verification. Model Driven Architecture [9] can be strict or relaxed, depending on the level of details in the model.

Human language is an informal language. Hence human language specification is informal, which is of low cost and flexible. In order for human language programming to work, we need to achieve precise specification using an informal language. We also need to maintain consistency across all modules.

In order for generative artificial language models to precisely create the software based on an informal specification, we suggest a structured specification. All statements in the specification are simple human language sentences. The structure organizes these sentences to achieve the expression ability for complex software modules. Since each sentence is short and simple, it is much easier for generative AI to correctly translate it into code. Errors caused by ambiguous sentences or translation errors can be easily caught and corrected. The code generated for each short human-language sentence can serve as a precise description.

The combination of simple human-language sentences, the code generated, and the structure of specification help us achieve human-language precise specification. To program in a human language is to write human-language specifications. We will naturally archive living documentation [10] since code and document are one.

In this article, we first propose a human-language module specification template. Then, we apply it to human-language UI test automation. Next, we discuss the object-oriented behavior

driven development methodology that supports the human-language module specification. We aim to demonstrate that software test scripts can be generated from human-language specifications.

We anticipate that further progress in this direction will lead to the programming of complex software in human language, making software development more intuitive and efficient.

II. EXECUTABLE HUMAN-LANGUAGE MODULE SPECIFICATION

A. Module's MAUSIE Model

A software **module** runs and interacts with the **users** (user can be other modules) through module **interface** in an **environment**. Inside the module, we use variables to hold the **states** of the module and **algorithms** responsible for computation. To make it easier for discussion, we use the first character of the six elements to form a word MAUSIE. See Figure 1 below.

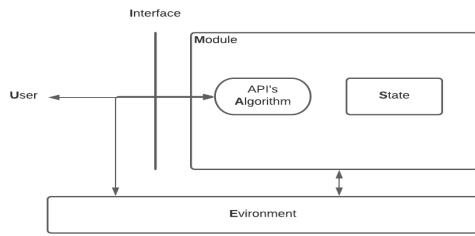


Fig. 1. MAUSIE Model Illustration

Module specification is a black-box specification. It does not define the internal state and algorithm of a module.

Module interface or API can be defined by a function name, inputs, outputs, and running environment.

Software interface specification defines the input and output relationship between a software program and its users. Software behavior specification goes a step further, detailing how the input and output relationship is affected by environmental factors. The module specification discussed here is a software behavior specification.

In the subsequent sections, we present an new approach to specifying software behavior through the description of variables (input, output and environment), interface functions, and the relationship of module interfaces (which are formed into a graph).

B. The Object Model of a Variable

A file to be uploaded is an input variable of the upload function. The free space on the computer is an environment variable, which could impact the upload function. User name and password are two input variables of the login function, and the login result is the output of this function.

To model the variables as objects, we can use a set of attributes to describe the characteristics of a variable. For each attribute dimension, we can then break down the attribute further into several situations, or equivalence classes. Finally, we can define a combination of the situations to reflect the business logic.

We consider the input variables, output variables, and environment variables as objects and use a standard object model template.

● Object Model Template

■ Attributes

◆ Attribute

● Equivalence Class

◆ More Attributes

■ States

◆ State

● List of attribute and equivalence class pairs

◆ More states

For example, if we want to describe a user name variable, we first use two attributes to describe the characteristics of the user name and some equivalence classes to define the details of each attribute. We use a YAML file to store the specification and a table (see an example in Table 1) to present the attributes and equivalence classes:

TABLE I. VARIABLE ATTRIBUTE MODELING

Attributes	Equivalence Classes
Length	User name length is less than 5 or greater than 10
	User name is between 5 and 10.
Status	User name is registered
	User name is not registered

Then, the states of a variable will be defined by some combination of the attributes and equivalence classes based on the business logic. We can also use a table (see an example in Table 2) to describe the combination of the attribute dimensions:

TABLE II. VARIABLE STATE MODELING

States	Length	Status
Case 1	User name length is less than 5 or greater than 10	User name is registered
Case 2	User name is between 5 and 10.	User name is registered
Case 3	User name length is less than 5 or greater than 10	User name is not registered
Case 4	User name is between 5 and 10.	User name is not registered

Invalid combination can be eliminated by conditional combination.

Environment variables can also be expressed this way. A browser for a Web application can have attributes of browser types (Chrome, Edge, Firefox) and browser versions (111.0.5563.147, ...).

C. CAR model for Interface and Interface Relationship Description

The behavior of an interface can be defined by three things: Condition, Action, Result.

An action presents a method or an event of the interface provided by either a module, a class, or a software. We use a word, a short sentence, or a sequence of short sentences to describe it.

A condition is a list of human-language statements describing the states of the input and environment variables involved. These conditions should be met when an action takes place.

A result is also a list of human-language statements describing the states of the output and environment variables involved. These results should be reached after an action takes place.

We use {Condition, Action and Result} (CAR) to describe an branch of the action to be described. It is similar to BDD's {Given, When, Then}. The difference is that BDD is procedure-oriented and use a set of workflows to define software's behavior. Ours is object-oriented and focus on the whole activity diagram.

We also use YAML file to store the specification.

- Action
 - CAR
 - ◆ Condition
 - Combination of equivalence classes
 - ◆ Result
 - Combination of equivalence classes
 - List of Next CARs
 - More CARs

To make it easier to understand and present visually, each action's CARs are grouped together under the action to which they belong.

We will use a table to present all CARs of an action. The relationship of modules is represented by a list of CARs in the Next CARs column. These next CARs define the permitted actions under this CAR's condition. As a result, we will get a graph of CARs, a type of activity diagram, for the related modules. The following table is a specification example of a Login function. See example in Table 3.

Action: Login, or a list of operations in human language.

TABLE III. ACTION MODELING

Condition	Result	Next CARs
● User name is between 5 and 10. ● User name is registered ● Password is valid	User reaches personal home page.	List of CARs of next API
... ..		

D. Module Specification Template

By assembling the above components together, we can get a template of module specification. This is an object model template.

- Object Model Template
 - Attributes
 - ◆ Attribute
 - Equivalence Class
 - ◆ More Attributes
 - States
 - ◆ State

- List of attribute and equivalence class pairs
- ◆ More states
- Actions
 - ◆ Action
 - CAR
 - Condition
 - ◆ Combination of equivalence classes
 - Result
 - ◆ Combination of equivalence classes
 - ◆ List of Next CARs
 - More CARs
 - More actions

When this template is used to describe a variable, the action section will not be used. When used to describe a module's interfaces, the attributes and states will not be used.

When used to describe a login web page, the page can have a user name text box, a password text box and a login button, we will use a module specification to describe the page, and three child objects to describe the three elements, user name textbox, password text box, and login button.

- Login Page Object
 - User Name object
 - ◆ Object model template
 - Password object
 - ◆ Object model template
 - Login button object
 - ◆ Object model template

E. CAR Graph

For every action, there are CARs specified (the Next CAR List in the object-oriented behavior specification template above). Each CAR can have Next CARs linked to it. All of the CARs of a software' modules specified form a directed graph. Each node of the graph is a CAR. See example in Figure 2.

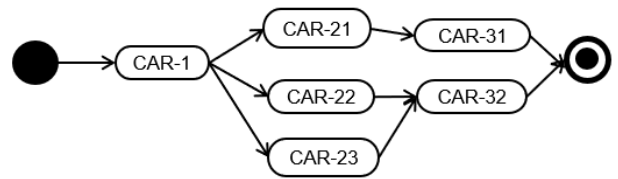


Fig. 2. CAR Graph

F. Discussion

The Class structure of a Java programming language holds a set of Java statements. Similar, the module specification is a structure that holds a set of human-language statements. The specification can be understood by both humans and machine.

This graph specifies all scenarios. So, it can be used to specify a software's behavior. We can also use Large Language Model or other AI models to "compiled" it into some high-level programming language source code. We can then get machine code using compilers or interpreters.

Each path in the graph represents a scenario in BDD or user story. See example in Figure 3.

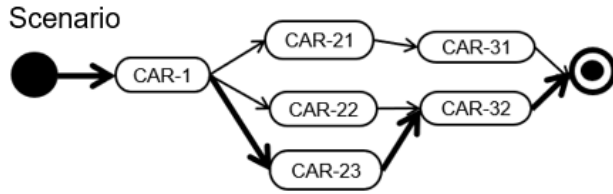


Fig. 3. A BDD Scenario In CAR graph

A test case is one scenario with test data defined. See example in Figure 4.

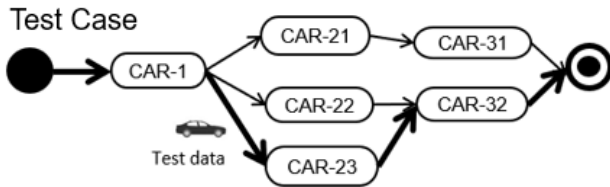


Fig. 4. A test case In CAR graph

A defect is a test case execution where actual behavior is different from defined behavior. See example in Figure 5.

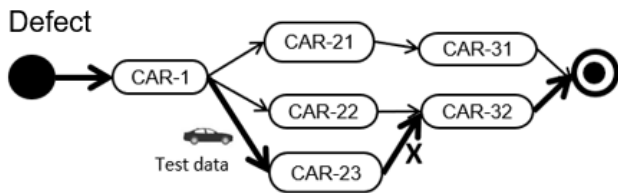


Fig. 5. A Defect In CAR Graph

A user guide is just one of the existing paths from two points on the CAR map, usually the shortest path from the start of the application to where the task is completed. See example in Figure 6.

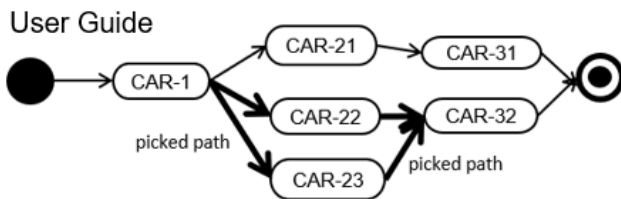


Fig. 6. A User Guide CAR Graph

These statements can be used as prompts for Large Language Model or other types of generative artificial intelligence models. Each prompt can be a simple statement, corresponding to a small number of lines of code. We can easily spot and correct errors caused by mistakes in the specification or generative AI models.

Unlike UML [11] and BPMN [12], the CAR diagram has few symbols and easy to learn.

III. EXAMPLE

We use Facebook as an example. Its interface is shown in Figure 7.

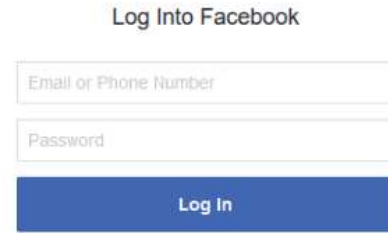


Fig. 7. Facebook Login Page

The feature is: “As a user, I want to log into the system using a name and a password”.

A user enters user name and password from the two text boxes and submits the request by clicking the “Log In” button.

The user name and password, and error message are treated as objects. Objects can be described using properties and actions in object-oriented methodology. The following Picture (Figure 8) shows how these are defined using SpecMap, our tool to support the approach. With the help of SpecMap, the object specification becomes simple.

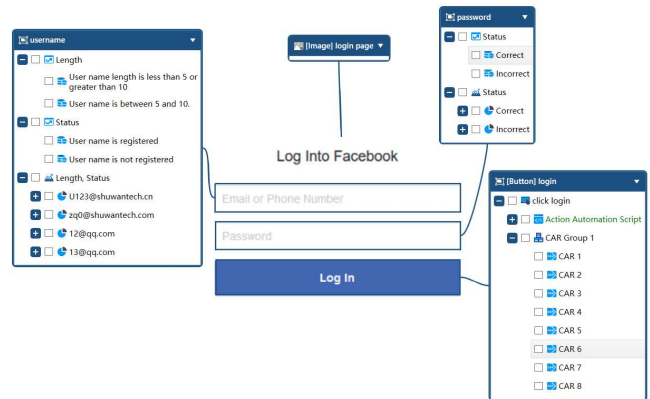


Fig. 8. Creating Login Page Specification In Specmap

After user logs in, the welcome page will be open, see the following picture (Figure 9). In this page, we specify what to assert as criteria that user as logged into Facebook.

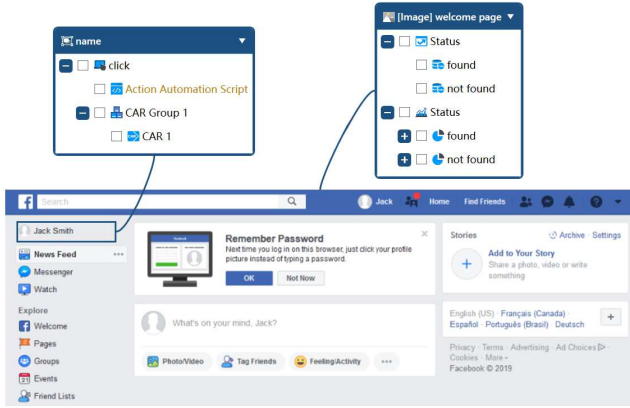


Fig. 9. Creating Homepage Specification

Single object combination analysis is accommodated by a dialog in SpecMap (See figure 10).

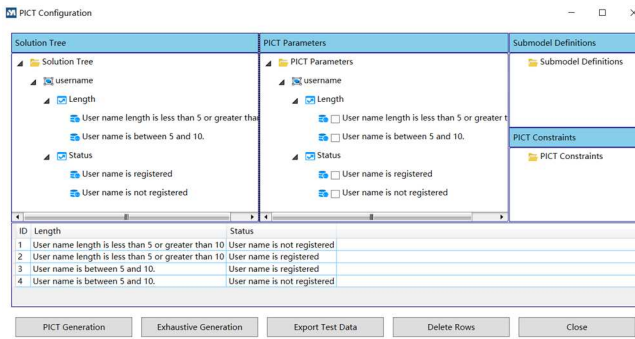


Fig. 10. Combination in SpecMap

With the help of SpecMap, test data are specified by equivalence class combinations. See Figure 11 below.

Test Data Details			
Length, Status		Variable	
Test Data Value	Data Source	Test Data Remark	Eq. Class
U123@shuwantech.cn			Length
zq0@shuwantech.com			Status

Fig. 11. Specifying Test Data in SpecMap

Action Login, its conditions and results can also be specified in SpecMap (See Figure 12)

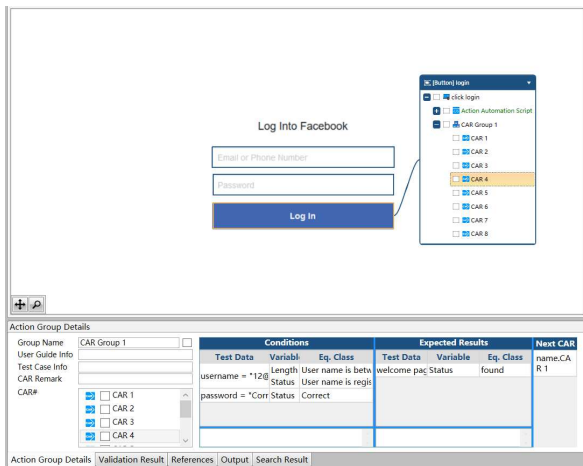


Fig. 12. Specifying Action in SpecMap

With the help of SpecMap, combination analysis of multiple inputs and environment variables in an action's condition can be done with the help of a dialog in SpecMap too (See Figure 13).

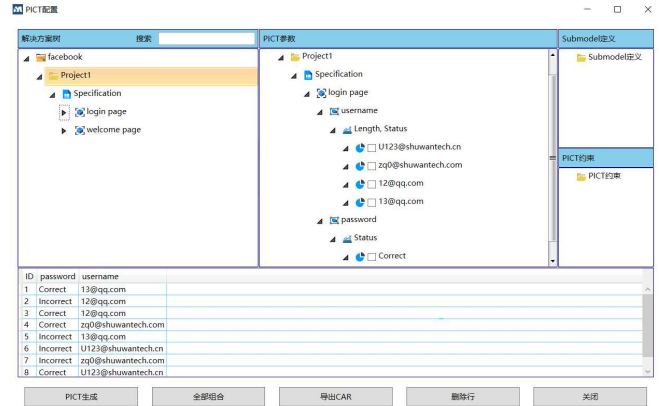


Fig. 13. Action Condition Combination Analysis in SpecMap

As a result of specifying the Login action, we get the CARs for the action (See Table 4).

TABLE IV. CARs IN SPECMAP

Name	Description
CAR 1	- username length is between 1 and 5 - username is registered - password is Incorrect
CAR 2	- username length is less than 5 or greater than 10 - username is not registered - password is Incorrect
CAR 3	- username length is less than 5 or greater than 10 - username is registered - password length is Incorrect
CAR 4	- username length is between 1 and 5 - username is registered - password is correct
CAR 5	- username length is between 1 and 5 - username is not registered - password is Incorrect
CAR 6	- username length is less than or greater than 10 - username is not registered - password is correct
CAR 7	- username length is between 1 and 5 - username is not registered - password is correct
CAR 8	- username length is less than 5 or greater than 10 - username is registered - password is correct

“click login 4” is for registered email and correct password; “click login 6, click login 7, click login 8” is for wrong user id. “click login 1” is for registered email and wrong password; “click login 2, click login 3, click login 5” is for registered phone number and wrong password.

Figure 14 is one of the manual test cases generated by SpecMap. It has two test steps:

1. User enter user id and pass word following the visual test
2. Expected Result is the open Welcome page.

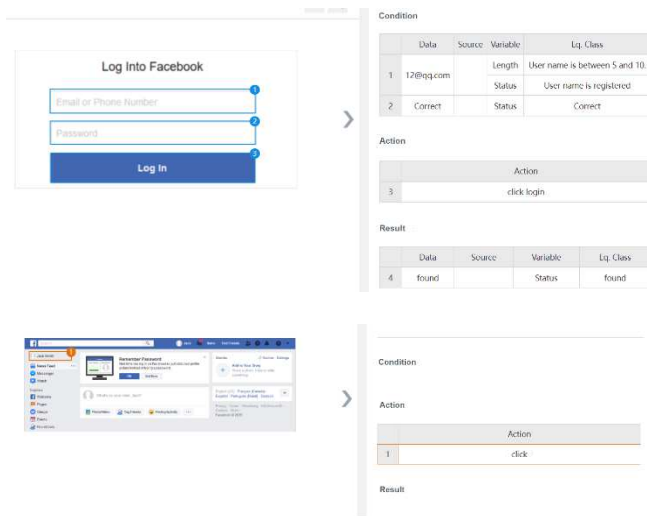


Fig. 14. Visual Manual Test Cases Created by SpecMap

In the above GUI, we see two text boxes: user name and password. The action login is implemented by the “Log In” button. In the API form, the action is implemented as the method “Login”. Its input arguments are the user name and password.

Boolean Login (String username, String password0{...}

OOBDD can be applied to GUI and API. Username and password are two objects. Login is the action.

A test set is a list of test cases. A test case is a list of steps. Each step is a CAR. Each CAR has three sections: Condition, Action, and Result. Condition is a list of getter, setter and assertion methods for the object’s test data in the condition section. Result is a list of getter, setter and assertion methods for the object’s test data in the result section. For text script, Action is a method that triggers the action call. For example, The function to be tested is Login. The action to test Login will be a method called TestLogin that calls Login function.

In OOBDD, the atomic building blocks of automation scripts will be: object’s Actions; object’s test data’s getter, data’s setter, or data’s assertion. We will get better structured test automation script. See illustration in Figure 15.



Fig. 15. OO Structure of Test Script

Figure 16 is an code example generated by SpecMap using generative AI. The automation framework used is OpenCV for non-intrusive test automation. We can also choose other frameworks, such as Selenium, Appium, etc. , or the mixed use of all of them.

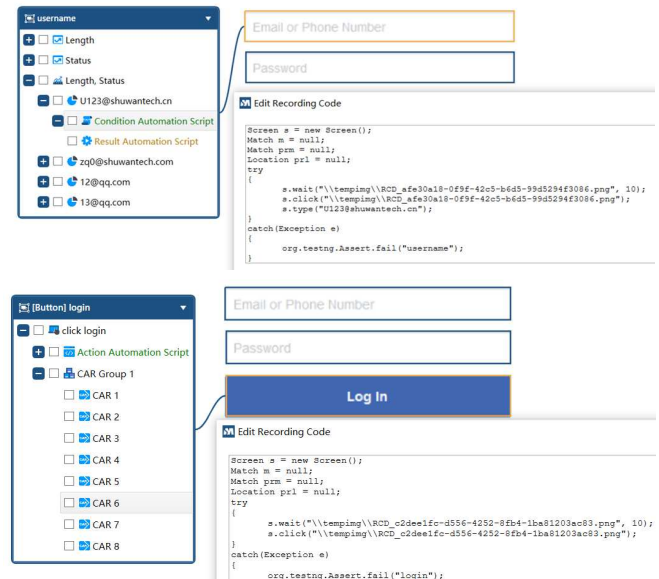


Fig. 16. Generated Test Script

The unification of manual test cases and automation script can be naturally realized in SpecMap, and in an object-oriented way.

IV. OBJECT-ORIENTED BEHAVIORE-DRIVEN DEVELOPMENT

Module behavior is the interaction between the module and the outside world at the interface of the module, plus the environment impact.

According to this definition, use cases, user stories, test cases and scenarios of behavior driven development (BDD) are all behaviors of the module under investigation. The difference lies in how the behavior is described.

Among the current approaches, we think BDD is a good starting point for human-language executable specification.

Use case, user story and BDD’s feature are all at the same level of abstraction and are procedure-oriented. Test cases are used to cover behavior details for use cases and user stories. Scenarios are used to cover details for BDD’s features. In BDD,

scenarios should be considered as acceptance criteria. It is not efficient to use BDD for full coverage of all edge cases. This is true to all procedure-oriented approaches.

Object-oriented, Google-Maps-style activity diagram and behavioral module specification differentiate OOBDD from BDD. The differences are highlighted in Table 5.

TABLE V. BDD vs, OOBDD

Area	BDD	OOBDD
Discovery	Write features	Write features
Formulation	Write scenarios using Given-When-Then structure	Object-oriented analysis
GUI Prototype	UI visualization is NOT involved in BDD scenarios.	Prototype is the interface of graphical applications. Its behavior will be analyzed. GUI is part of the OO Behavior specification. But it does not create maintenance issues.
Code generation	Using GPT, the software implementing the specification can be generated but it is not easy to cover the edge cases.	Using GPT, the software implementing the specification can be generated. The edge cases are specified and be implemented.
Test Case Generation	Test cases and scenarios are the same and created at the same time.	Objects are specified in behavior analysis. Test cases are created when test execution is needed. Just-In-Time test case creation will ease test case maintenance.
Manual Testing	Scenarios can be used as manual test cases but test cases are hard to read and automation is their focus.	Great visual manual test cases.
Automation	Procedure-oriented test case automation	Object-oriented test case automation

V. CONCLUSION AND FUTURE WORK

Object Oriented Behavior Driven Development approach integrates several existing methodologies. We view variables as objects. We define an API's behavior using BDD's style. We use YAML specification style of OpenAPI for documentation.[13] We use equivalence classes, combination and graph theory to fully cover all scenarios. We use tables to

visualize the specification details.[4] Human-language module specification is the result of OOBDD. This specification can be used for many things, such as test case creation, user guide creation, and workflow diagram. It works perfectly with Generative AI to achieve human-language test automation for complex software. We maintain behavior specifications, not test cases. The request and resource allocation of timely updated test scenarios and by all companies will naturally realize living documentation.

With the tools created, the specification creation and maintenance work becomes simple and efficient.

If Generative AI can be our programmer, it should find all information needed in the module specifications. We believe OOBDD can work with Generative AI to achieve human-language programming. Experiment and tool creation is underway.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, Ł. Kaiser, and I. Polosukhin., "Attention is All You Need" , NIPS'17: Proceedings of the 31st International Conference on Neural Information Processing Systems December 2017 Pages 6000–6010
- [2] Systems and software engineering – Methods and tools for model-based systems and software engineering, ISO_IEC_IEEE FDIS 24641
- [3] Bertrand Meyers, "What Do ChatGPT and AI-based Automatic Program Generation Mean for the Future of Software", <https://cacm.acm.org/blogs/blog-cacm/268103-what-do-chatgpt-and-ai-based-automatic-program-generation-mean-for-the-future-of-software/fulltext>
- [4] D. L. Parnas, Tabular Representation of Relations, CRL Report 260, Telecommunications Research Institute of Ontario (TRIO), McMaster University, Hamilton, Ontario, Canada, 1992.
- [5] D. L. Parnas, J. Madey, "Functional documents for computer systems", Science of Computer Programming 25 (1995) 41-61
- [6] Kent Beck, "Test-Driven Development", https://en.wikipedia.org/wiki/Test-driven_development
- [7] D. North, "Introducing BDD", <https://dannorth.net/introducing-bdd/>
- [8] J. F. Smart, "BDD in Action: Behavior-driven development for the whole software lifecycle", Manning Publications. ISBN 9781617291654.
- [9] OMG Standards Development Organization, "MDA - The Architecture Of Choice For A Changing World", <http://www.omg.org/mda/>
- [10] Cyrille Martraire, "Living Documentation: Continuous Knowledge Sharing by Design", <https://www.amazon.com/Living-Documentation-Cyrille-Martraire/dp/0134689321>
- [11] Unified Modeling Language, <http://www.uml.org/>
- [12] Business Process Model and Notation , <https://www.bpmn.org/>
- [13] OpenAPI Specification , <https://spec.openapis.org/oas/latest.html>